

ROS 通信机制实验报告

人工智能 2302 蒋梓轩
人工智能 2302 汪翌秦
人工智能 2302 孙靖淞

2026 年 4 月 28 日

1 实验目的

本次实验围绕 ROS Noetic 中两种最常见的通信方式——话题 (Topic) 和服务 (Service) 展开，希望通过自己实现节点、运行节点、再用工具观察节点之间的交互，建立起对 ROS 计算图的直观理解。在 ROS 里，机器人系统通常被拆成多个职责单一的节点，节点之间不直接调用对方函数，而是依靠 ROS Master 的注册机制完成对接。这种“分布式 + 解耦”的设计是后续学习感知、规划、控制等模块的基础，因此本次实验的目标就是把这套机制从概念落实到可运行的代码上。

具体来说，实验分三块内容。第一块是话题通信，需要写一个以固定频率发送字符串的发布者，再用一个订阅者把消息接收并打印出来，借此感受 ROS 中数据流的异步特性。第二块是服务通信，由客户端提交两个整数，服务端完成求和后返回结果，体会请求-响应这种同步式调用与话题通信的差异。第三块是 turtlesim 仿真：通过向速度话题发布 `geometry_msgs/Twist` 让小乌龟做圆周运动，再订阅 `/turtle1/pose` 拿到位置和姿态信息。所有这些节点都通过 launch 文件统一管理，体会 `roslaunch` 在多节点工程中的作用。

2 ROS 通信机制概述

ROS 之所以能把一个机器人系统模块化地组织起来，关键在于它把“功能”和“通信”解耦。每个节点只关心自己负责的一件事——比如发布速度命令、订阅雷达扫描、提供求逆解服务等等——至于谁与谁通信、用什么协议，则由 ROS Master 在节点启动时统一登记。需要注意的是，Master 只承担“撮合”角色：它记录每个节点的名字、发布的话题、订阅的话题、提供的服务，并把这些信息提供给查询方；真正的数据并不经过 Master 中转，而是节点之间直接建立连接传输。这种架构的好处很明显——单个节点崩溃不会拖垮整个系统，节点也可以分布在不同主机上运行。

通信双方真正能“听懂彼此”的前提是消息类型一致。话题通信使用 `.msg` 文件描述的消息结构，例如本实验中的字符串收发用到 `std_msgs/String`，速度命令用到 `geometry_msgs/Twist`。服务通信则用 `.srv` 文件，把“请求”和“响应”两段数据放在同一个文件里，本实验自定义的 `AddInts` 服务就是请求两个整数、响应一个求和结果。

ROS 中常用的通信机制有三种：话题、服务、参数。三者解决的问题并不相同——持续产生的数据流走话题最合适，明确的“问一答一”任务用服务更直接，而那些少量稳定不变的配置量则放在参数服务器中读取一次即可。三者的对比可以参考表 1。

表 1: ROS 常见通信机制对比

机制	通信方式	典型特点	适用场景
Topic	发布/订阅	异步、多对多、连续数据流	传感器数据、速度命令、状态广播
Service	请求/响应	同步、一对一、一次调用一次返回	查询、计算、状态切换
Parameter	参数服务器	保存配置，读取频率通常较低	节点参数、实验配置

本次实验主要练习了话题和服务两种机制。字符串发布订阅、`turtlesim` 速度控制都属于话题通信，节点之间只要约定好话题名和消息类型就能建立松耦合连接；自定义 `AddInts` 服务则属于服务通信，调用流程更接近一次远程函数调用。`launch` 文件本身不算新的通信机制，但它能把一组节点、参数和命名规则打包到一起，让多节点实验的复现变得简单。

3 实验步骤与结果分析

3.1 环境与工作空间准备

实验环境用的是 ROS Noetic。在创建 `catkin` 工作空间之前先 `source` 一次 ROS 安装目录下的 `setup.bash`，让终端能识别 ROS 命令。工作空间结构遵循标准 `catkin` 约定，`src/` 目录下放三个功能包：发布订阅、服务通信、小乌龟控制。每次新建或修改包之后都要执行 `catkin_make`，再 `source` 工作空间下的 `devel/setup.bash`，使新包名、Python 节点和自定义服务类型能被 ROS 工具识别。

由于本实验所有节点都用 Python 实现，`scripts/` 下的每个 `.py` 文件都需要在首行写明 `#!/usr/bin/env python3`，并用 `chmod +x` 赋予执行权限。服务通信功能包额外依赖 `message_generation` 和 `message_runtime`：前者负责把 `.srv` 文件编译成可被 `rospy` 导入的 Python 模块，后者声明运行期对生成消息的依赖。

3.2 小乌龟圆周运动与位姿获取

`turtlesim` 是 ROS 自带的二维仿真工具，可以直观地观察发布者和订阅者之间的数据流是怎样驱动一个仿真对象运动的。实验中先启动 `turtlesim_node`，再发布速度指令 `geometry_msgs/Twist` 到 `/turtle1/cmd_vel`。当线速度持续保持为 1.0、角速度持续保持为 0.5 时，小乌龟会因为前进的同时不断转弯而画出圆形轨迹。

之所以选这两个值，是因为它们刚好能让圆形清晰可见且转动不至于太快——线速度决定每秒前进多少单位，角速度决定每秒转动多少弧度，二者比例越大圆越大。仿真节点收到速度后会以高频率（约 62 Hz）更新内部位姿，因此即使我们以 10 Hz 的频率发布命令，运动看起来仍然是连续平滑的。

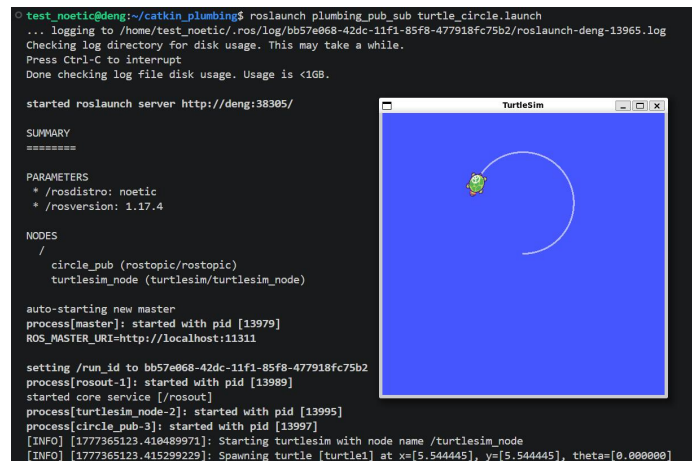
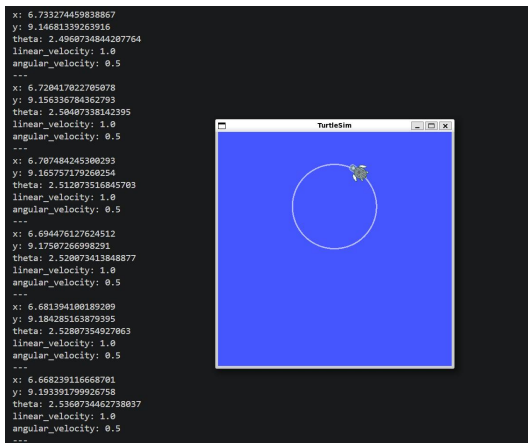


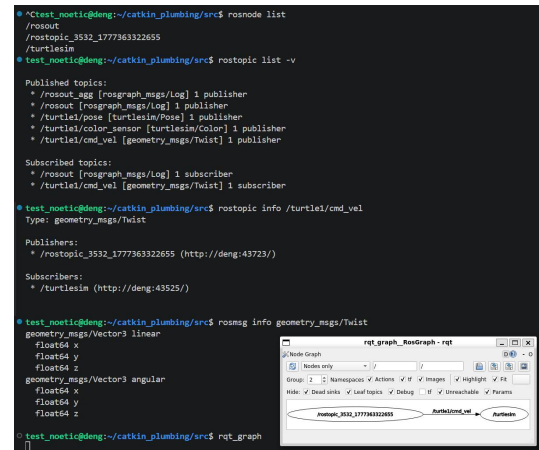
图 1：用 `launch` 文件启动 `turtlesim` 与速度发布节点后小乌龟开始作圆周运动。

仅看到圆形并不足以验证仿真状态，所以我们又订阅了 `/turtle1/pose`。这个话题由 `turtlesim` 内部以高频率发布，包含 x 、 y 、 θ 、`linear_velocity`、`angular_velocity` 五个字段。订阅节点收到后将这些值打印出来，可以看到 x 、 y 在持续变化、 θ 均匀增加、线速度与角速度稳定在我们设定的数值附近，这与圆周运动的运动学特征完全一致。

为了从计算图层面进一步验证整个系统的连接关系，我们用 `rostopic list` 查看活动节点、用 `rostopic list -v` 列出话题及其订阅发布情况、用 `rostopic info /turtle1/cmd_vel` 查看具体话题的拓扑、再用 `rosmmsg info geometry_msgs/Twist` 查看消息字段。这些信息合起来就构成 ROS



(a) 圆周运动与位姿输出



(b) 节点、话题与 rqt_graph

图 2: 小乌龟实验中位姿反馈与节点关系图。

计算图的完整描述。rqt_graph 是它的可视化版本，从图 2b 可以清楚看到 circle_pub 节点通过 /turtle1/cmd_vel 把速度指令传给 turtlesim_node，节点之间是松耦合的，连接关系完全由话题名定义。

3.3 话题通信实验

话题通信采用的是发布/订阅模式。发布方不需要知道有谁在订阅，只是把消息扔到指定话题上；订阅方也不必关心是谁发布的，只要订阅同名话题就能在回调中收到数据。这种解耦特性非常适合传感器数据、速度命令、状态广播之类的连续数据流。

3.3.1 发布者

发布者节点初始化时使用名字 talker_py，向 chatter 话题发布 std_msgs/String 类型的消息。循环频率由 rospy.Rate(10) 控制，意味着每秒大约执行 10 次循环、发出 10 条消息。每条消息的内容是字符串 Hello ROS Python 加上自增的整数计数，发布之后同时打印到终端日志里方便观察。

3.3.2 订阅者

订阅者节点叫 listener_py，使用 rospy.Subscriber 订阅 chatter 话题，并指定回调函数读取 msg.data 字段。从图 3b 可以看到，订阅终端持续打印“我听见：Hello ROS Python ...”，与发布端的内容完全对应。由于发布频率为 10 Hz，订阅端的输出节奏也大致是 0.1 秒一行，二者频率上保持一致。

为了再现“多节点一键启动”的工作方式，我们还为这一对节点写了 launch 文件。运行 roslaunch 之后，终端先打印节点摘要，然后发布与订阅日志开始交替出现，这说明两个节点已经被同一个 launch 文件托管并在同一个 ROS Master 下完成了通信。

值得指出的是，订阅节点不论是用 rosrn 单独启动还是被 launch 文件一起拉起来，只要订阅的话题名一致就能正常收到数据。这正是话题通信灵活性的体现：节点之间靠话题名约定耦合，与启动方式无关。

3.4 服务通信实验

服务通信和话题通信最大的区别在于它是同步的——客户端发起请求后会阻塞，等服务端处理完返回结果才继续往下走。本实验我们自定义了一个 AddInts 服务，请求字段是 num1 和 num2 两个整数，响应字段是它们的和 sum。

服务端节点 addints_server_py 启动后用 rospy.Service("AddInts_py", AddInts, handle)


```
[INFO] [1777365350.469453]: 发送的消息: Hello ROS Python 7
[INFO] [1777365350.569489]: 发送的消息: Hello ROS Python 8
[INFO] [1777365350.669513]: 发送的消息: Hello ROS Python 9
[INFO] [1777365350.769356]: 发送的消息: Hello ROS Python 10
[INFO] [1777365350.869546]: 发送的消息: Hello ROS Python 11
[INFO] [1777365350.969503]: 发送的消息: Hello ROS Python 12
[INFO] [1777365351.069476]: 发送的消息: Hello ROS Python 13
[INFO] [1777365351.169474]: 发送的消息: Hello ROS Python 14
[INFO] [1777365351.269539]: 发送的消息: Hello ROS Python 15
[INFO] [1777365351.369502]: 发送的消息: Hello ROS Python 16
[INFO] [1777365351.469801]: 发送的消息: Hello ROS Python 17
[INFO] [1777365351.569516]: 发送的消息: Hello ROS Python 18
[INFO] [1777365351.669498]: 发送的消息: Hello ROS Python 19
[INFO] [1777365351.769518]: 发送的消息: Hello ROS Python 20
[INFO] [1777365351.869517]: 发送的消息: Hello ROS Python 21
[INFO] [1777365351.969505]: 发送的消息: Hello ROS Python 22
[INFO] [1777365352.069573]: 发送的消息: Hello ROS Python 23
[INFO] [1777365352.169523]: 发送的消息: Hello ROS Python 24
[INFO] [1777365352.269487]: 发送的消息: Hello ROS Python 25
[INFO] [1777365352.369632]: 发送的消息: Hello ROS Python 26
[INFO] [1777365352.469518]: 发送的消息: Hello ROS Python 27
[INFO] [1777365352.569497]: 发送的消息: Hello ROS Python 28
[INFO] [1777365352.669485]: 发送的消息: Hello ROS Python 29
[INFO] [1777365352.769494]: 发送的消息: Hello ROS Python 30
```

(a) 发布者节点以 10 Hz 发出消息

```
[INFO] [1777365409.222440]: 我听见: Hello ROS Python 566
[INFO] [1777365409.322515]: 我听见: Hello ROS Python 567
[INFO] [1777365409.422569]: 我听见: Hello ROS Python 568
[INFO] [1777365409.522714]: 我听见: Hello ROS Python 569
[INFO] [1777365409.622706]: 我听见: Hello ROS Python 570
[INFO] [1777365409.722553]: 我听见: Hello ROS Python 571
[INFO] [1777365409.822977]: 我听见: Hello ROS Python 572
[INFO] [1777365409.922698]: 我听见: Hello ROS Python 573
[INFO] [1777365410.022756]: 我听见: Hello ROS Python 574
[INFO] [1777365410.122480]: 我听见: Hello ROS Python 575
[INFO] [1777365410.222629]: 我听见: Hello ROS Python 576
[INFO] [1777365410.322584]: 我听见: Hello ROS Python 577
[INFO] [1777365410.422621]: 我听见: Hello ROS Python 578
[INFO] [1777365410.522513]: 我听见: Hello ROS Python 579
[INFO] [1777365410.622571]: 我听见: Hello ROS Python 580
[INFO] [1777365410.722580]: 我听见: Hello ROS Python 581
[INFO] [1777365410.823055]: 我听见: Hello ROS Python 582
[INFO] [1777365410.922590]: 我听见: Hello ROS Python 583
[INFO] [1777365411.022502]: 我听见: Hello ROS Python 584
[INFO] [1777365411.122495]: 我听见: Hello ROS Python 585
[INFO] [1777365411.222554]: 我听见: Hello ROS Python 586
[INFO] [1777365411.322515]: 我听见: Hello ROS Python 587
[INFO] [1777365411.422730]: 我听见: Hello ROS Python 588
[INFO] [1777365411.522613]: 我听见: Hello ROS Python 589
```

(b) 订阅者节点接收并打印

图 3: 话题通信中发布与订阅节点的运行结果。

```
test_noetic@denq:~/catkin_plumbing$ roslaunch plumbing_pub_sub start_pub_sub.launch
... logging to /home/test_noetic/.ros/log/d8b9b526-42d9-11f1-85f8-477918fc73b2/roslaunch-deng-7398.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <10B.

started roslaunch server http://deng:38281/

SUMMARY
=====
PARAMETERS
 * /rostdistro: noetic
 * /rosversion: 1.17.4

NODES
 /
  pub_node (plumbing_pub_sub/demo01_pub_node)
  sub_node (plumbing_pub_sub/demo02_sub_node)

auto-starting new master
process[master]: started with pid [7412]
ROS_MASTER_URI=http://localhost:11311

setting /run_id to d8b9b526-42d9-11f1-85f8-477918fc73b2
process[roscout-1]: started with pid [7422]
started core service [/roscout]
process[pub_node-2]: started with pid [7426]
process[sub_node-3]: started with pid [7430]
[INFO] [1777363892.497194392]: 发送的消息: Hello ROS 0
[INFO] [1777363892.597239958]: 发送的消息: Hello ROS 1
[INFO] [1777363892.697234902]: 发送的消息: Hello ROS 2
[INFO] [1777363892.797257244]: 发送的消息: Hello ROS 3
[INFO] [1777363892.797568732]: 我听见: Hello ROS 3
[INFO] [1777363892.897569848]: 发送的消息: Hello ROS 4
[INFO] [1777363892.897569710]: 我听见: Hello ROS 4
[INFO] [1777363892.997329158]: 发送的消息: Hello ROS 5
[INFO] [1777363892.997315194]: 我听见: Hello ROS 5
[INFO] [1777363893.097289170]: 发送的消息: Hello ROS 6
[INFO] [1777363893.097575897]: 我听见: Hello ROS 6
[INFO] [1777363893.197279597]: 发送的消息: Hello ROS 7
[INFO] [1777363893.197476544]: 我听见: Hello ROS 7
```

(a) 用 launch 启动发布订阅节点

```
test_noetic@denq:~/catkin_plumbing$ rosrn plumbing_pub_sub demo02_sub_node
[INFO] [1777363826.169279256]: 我听见: Hello ROS 88
[INFO] [1777363826.269218380]: 我听见: Hello ROS 89
[INFO] [1777363826.369198607]: 我听见: Hello ROS 90
[INFO] [1777363826.469094189]: 我听见: Hello ROS 91
[INFO] [1777363826.569109950]: 我听见: Hello ROS 92
[INFO] [1777363826.669097631]: 我听见: Hello ROS 93
[INFO] [1777363826.769115731]: 我听见: Hello ROS 94
[INFO] [1777363826.869110196]: 我听见: Hello ROS 95
[INFO] [1777363826.969117441]: 我听见: Hello ROS 96
[INFO] [1777363827.069099630]: 我听见: Hello ROS 97
[INFO] [1777363827.169268044]: 我听见: Hello ROS 98
[INFO] [1777363827.269221161]: 我听见: Hello ROS 99
[INFO] [1777363827.369044054]: 我听见: Hello ROS 100
[INFO] [1777363827.469140023]: 我听见: Hello ROS 101
[INFO] [1777363827.569227344]: 我听见: Hello ROS 102
[INFO] [1777363827.669208500]: 我听见: Hello ROS 103
[INFO] [1777363827.769205399]: 我听见: Hello ROS 104
[INFO] [1777363827.869346435]: 我听见: Hello ROS 105
[INFO] [1777363827.969151887]: 我听见: Hello ROS 106
[INFO] [1777363828.069172427]: 我听见: Hello ROS 107
[INFO] [1777363828.169293555]: 我听见: Hello ROS 108
[INFO] [1777363828.269153184]: 我听见: Hello ROS 109
[INFO] [1777363828.369192798]: 我听见: Hello ROS 110
[INFO] [1777363828.469174999]: 我听见: Hello ROS 111
[INFO] [1777363828.569274985]: 我听见: Hello ROS 112
[INFO] [1777363828.669380950]: 我听见: Hello ROS 113
[INFO] [1777363828.769140338]: 我听见: Hello ROS 114
[INFO] [1777363828.869115204]: 我听见: Hello ROS 115
[INFO] [1777363828.969236438]: 我听见: Hello ROS 116
[INFO] [1777363829.069119594]: 我听见: Hello ROS 117
[INFO] [1777363829.169186695]: 我听见: Hello ROS 118
```

(b) 单独运行的订阅节点接收消息

图 4: 发布订阅实验在 launch 启动与单独运行下的结果。

注册服务并阻塞等待请求。处理回调 `handle` 收到请求后，先打印两个数据，然后做合法性判断：如果有负数就记录错误日志并返回默认响应，否则返回二者之和。从图 5a 可以看到服务端正确接收到了 10 和 20 的请求。

客户端 `addints_client_py` 在调用之前先用 `rospy.wait_for_service("AddInts_py")` 等待服务出现，再用 `rospy.ServiceProxy` 创建一个服务代理。这种“先等再调”的写法避免了启动顺序错乱导致的失败：即便客户端跑起来时服务端还没注册完毕，客户端也只是阻塞等待而不会立刻报错。客户端提交 10 和 20 之后收到了 30，整个调用链路完整跑通（图 5b）。

```
test_noetic@deng:~/catkin_plumbing$ rosrn plumbing_server_client demo01_server.py
[INFO] [1777365548.061613]: Python 服务已经启动...
[INFO] [1777365608.828102]: 服务器收到请求: num1 = 10, num2 = 20
```

(a) 服务端收到 10 与 20

```
test_noetic@deng:~/catkin_plumbing/src/plumbing_server_client$ rosrn plumbing_server_client demo02_client.py 10 20
[INFO] [1777365608.829781]: 响应结果: 30
```

(b) 客户端取到求和结果 30

图 5: 服务通信中服务端与客户端的运行结果。

除了 Python 实现，我们也在课程参考代码上跑了一遍 C++ 版本的服务通信，命令行结果同样显示“请求正常处理，响应结果：30”（图 6）。两种语言下结果一致，再次说明服务通信的契约只取决于服务名和服务类型——对调用方而言，背后用什么语言实现并不影响最终结果。

```
test_noetic@deng:~/catkin_plumbing$ rosrn plumbing_server_client demo02_client_node 10 20
[INFO] [1777364977.318711435]: 请求正常处理，响应结果: 30
```

图 6: 基本服务通信实验中客户端发送 10 与 20，服务端返回 30。

3.5 launch 文件实验

当一次实验涉及多个节点时，逐个用 `rosrn` 启动既容易遗漏也不便于复现。launch 文件就是为了解决这件事——它用 XML 把要启动的节点、参数和命名规则全部写下来，再交给 `roslaunch` 统一管理。本实验分别为小乌龟仿真和发布订阅写了 launch 文件。

小乌龟实验的 launch 同时启动 `turtlesim_node` 和 `circle_pub` 两个节点。`roslaunch` 检测到没有正在运行的 master 时会先自启动一个 `roscore`，然后按照 launch 顺序拉起两个节点；只要仿真窗口出现，小乌龟立刻开始画圆。发布订阅实验的 launch 也是类似——“发布者打印”“发送的消息”、“订阅者打印”“我听见”，两条日志在终端中交替出现，一目了然地说明它们被绑定到了同一个 ROS 计算图里。

4 结论

通过本次实验我们认识到，ROS 中节点之间的协作并不依赖代码层面的直接调用，而是依靠注册中心（Master）和统一的通信图。话题通信支撑了 `chatter` 字符串收发、`turtlesim` 的速度控制和位姿订阅，这些场景都体现了它在连续数据流上的优势；服务通信支撑了 `AddInts` 整数求和，体现的是它一次调用一次返回的同步特性；launch 文件则把多节点的启动过程一次性固化下来，方便实验复现和分发。

从运行结果可以看出，只要话题名、服务名和消息类型对得上，节点用 `rosrn` 单独启动还是 `roslaunch` 一并启动都能正常通信，节点的语言（Python 或 C++）也不影响最终结果。这次实验完成了从单节点编写到多节点协作的过渡，也为后续做更复杂的机器人系统打好了基础。

5 问题讨论

节点命名、launch 命名与编译目标命名的区别。`init_node()` 中传入的字符串是节点向 Master 注册的运行期名称，`rostopic list`、`rqt_graph` 看到的都是它。launch 文件 `<node>` 标签里的 `name` 同样是运行期名，但由 `roslaunch` 在启动时统一指定，会覆盖代码里的默认值，便于同一份代码在不同 launch 中被赋予不同节点名。CMakeLists.txt 里 `add_executable` 中的名字属于编译层面，只决定生成哪个可执行文件，`rosrn` 用它定位二进制，本身不是计算图中的节点名。

一个功能包可以包含多少节点。没有数量限制：发布者、订阅者、服务端、客户端可以共存在同一个包里，本实验的发布订阅包和服务包都包含两个节点。实际开发中更看重组织合理性——把功能无关的节点塞进同一个包虽然能编过，但维护和依赖都会变乱，更常见的做法是按“一组紧密相关的功能”划分包的边界。

roslaunch 是否需要提前启动 roscore。多数情况下不需要。roslaunch 启动时会找当前可用的 Master，找不到就自动拉起一个新的 roscore；如果系统里已经有正在运行的 master 或显式设置了 ROS_MASTER_URI，则连接到对应实例而不会重复启动。手动跑 roscore 仍可行，只是非必需。

话题自定义消息与服务自定义消息的比较。话题消息保存在 .msg 文件里，描述一种单向数据流的结构，发送方不关心接收方是否处理完，适合传感器数据、位姿、控制指令的连续输出。服务消息保存在 .srv 文件里，用 --- 分成请求和响应两块，描述一次调用的输入输出关系，本质上更像远程函数调用。

为什么 CMakeLists.txt 中需要 add_dependencies()。功能包里一旦使用了自定义消息或服务，构建过程必须先把 .msg/.srv 转成对应语言的代码（C++ 是头文件如 AddInts.h）。如果可执行目标在头文件未生成时就开始编译，会报“找不到头文件”。add_dependencies() 的作用就是显式约束编译顺序，它不改变程序逻辑。Python 节点因为 rospy 在运行期动态导入消息模块，不存在这种依赖。

先启动客户端再启动服务端的问题。客户端启动后立即发起调用而服务端还没注册，同步调用会因 Master 上找不到提供者而失败。解决方法有两种：按顺序启动，或在客户端代码里加等待逻辑（C++ 用 ros::service::waitForService()，Python 用 rospy.wait_for_service()），本实验客户端采用了后者。如果服务端始终未起，客户端会一直阻塞除非设超时；服务端崩溃后再次调用会抛异常，需要异常捕获并决定是否重试。类似思路也体现在话题通信的 queue_size 上——队列太小丢老消息，太大增加延迟。最后，rospy.spin() 适合“被动等待回调”的订阅者和服务端，while not rospy.is_shutdown() 更适合“主动周期性执行”的发布者。